

Tarea 2: Números Pseudoaleatorios y Métodos de Monte Carlo

Didvin Nohel Estrada Pineda

Carné 14092

Curso Libre de Configuración 2: Análisis de Datos

Universidad del Istmo de Guatemala

Septiembre 2025

1. Generadores de números pseudoaleatorios

Para esta primera parte se implementaron cinco algoritmos generadores de pseudoaleatorios directamente en Python, sin apoyarse en `numpy.random` ni en el módulo `random` como motor. Cada implementación recibe su semilla como argumento explícito, y las semillas usadas en las figuras de este documento quedan documentadas en cada subsección y dentro del código fuente correspondiente en `src/prngs/`.

1.1. Generadores de congruencia lineal (LCG)

Un LCG produce una secuencia de enteros mediante la recurrencia

$$x_{n+1} = (ax_n + c) \text{ mód } m,$$

y normaliza cada término como $u_n = x_n/m \in [0, 1)$. La elección de a , c y m determina por completo la calidad del generador: el teorema de Hull–Dobell establece que el LCG alcanza período completo m si y solo si $\gcd(c, m) = 1$, $a - 1$ es múltiplo de todos los factores primos de m , y $a - 1$ es múltiplo de 4 cuando m es múltiplo de 4.

Algorithm 1 Generador de congruencia lineal

Require: semilla x_0 , multiplicador a , incremento c , módulo m

```
1: for  $n = 0, 1, 2, \dots$  do  
2:    $x_{n+1} \leftarrow (a \cdot x_n + c) \text{ mód } m$   
3:    $u_{n+1} \leftarrow x_{n+1}/m$   
4: end for
```

Se usaron los parámetros de Numerical Recipes $a = 1664525$, $c = 1013904223$, $m = 2^{32}$, que cumplen las condiciones de Hull–Dobell y por lo tanto alcanzan período completo 2^{32} . La semilla utilizada en las figuras fue $x_0 = 20250919$.

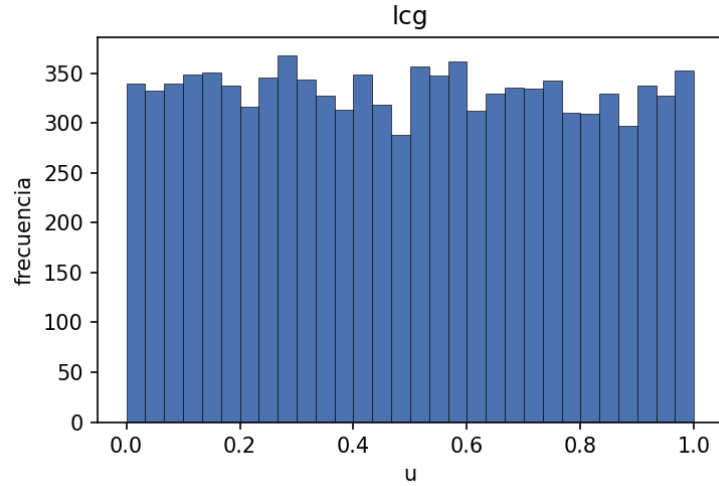


Figura 1: Histograma de 10,000 valores generados con el LCG, normalizados a $[0, 1]$.

Con $N = 10,000$ valores, la prueba chi-cuadrado de bondad de ajuste (20 categorías, 19 grados de libertad) dio un estadístico $\chi^2 = 28,39$ con $p = 0,076$: al ser mayor que 0,05 no se rechaza la hipótesis de uniformidad. La autocorrelación de retardo 1 fue $r = 0,0022$ con $p = 0,824$, es decir, no hay evidencia de dependencia serial entre valores consecutivos.

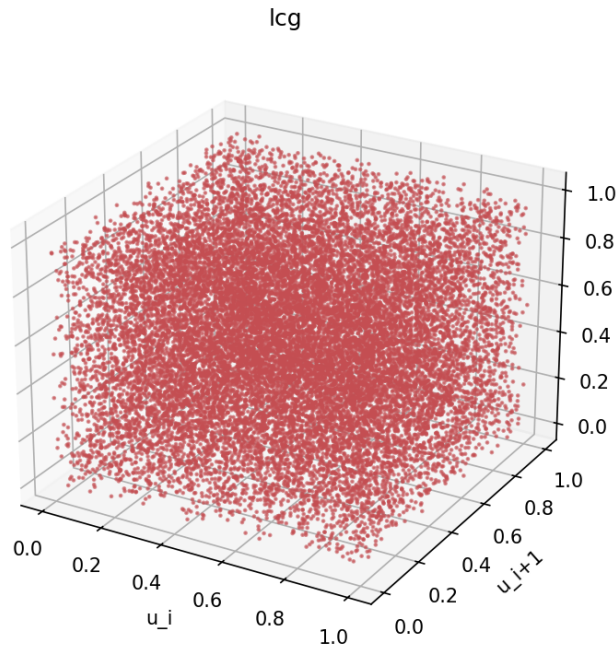


Figura 2: Ternas consecutivas (u_i, u_{i+1}, u_{i+2}) del LCG en el cubo $[0, 1]^3$.

Las ternas del LCG llenan el cubo de manera homogénea desde cualquier ángulo de observación, sin estructura visible. Esto es consistente con haber elegido parámetros que satisfacen Hull–Dobell: un buen LCG no debería mostrar ningún patrón geométrico en la prueba espectral.

1.2. Método de cuadrados medios

El método de cuadrados medios, propuesto por von Neumann, eleva al cuadrado la semilla actual y extrae los dígitos centrales como el siguiente término de la secuencia. Con d dígitos por término, si x_n tiene d dígitos, x_n^2 tiene hasta $2d$ dígitos, y x_{n+1} se forma tomando los d dígitos centrales de ese cuadrado (rellenando con ceros a la izquierda si hace falta).

Algorithm 2 Método de cuadrados medios

Require: semilla x_0 de d dígitos

- 1: **for** $n = 0, 1, 2, \dots$ **do**
 - 2: $y \leftarrow x_n^2$, escrito con $2d$ dígitos (rellenando con ceros a la izquierda)
 - 3: $x_{n+1} \leftarrow$ los d dígitos centrales de y
 - 4: $u_{n+1} \leftarrow x_{n+1}/10^d$
 - 5: **end for**
-

No existe una fórmula cerrada para el período de este método: al ser un mapeo determinista sobre un conjunto finito de 10^d estados, eventualmente entra en un ciclo, pero ese ciclo suele ser mucho más corto que 10^d , y en muchos casos degenera rápidamente a 0 o a ciclos muy cortos. Se usó $d = 8$ dígitos y semilla $x_0 = 57282719$.

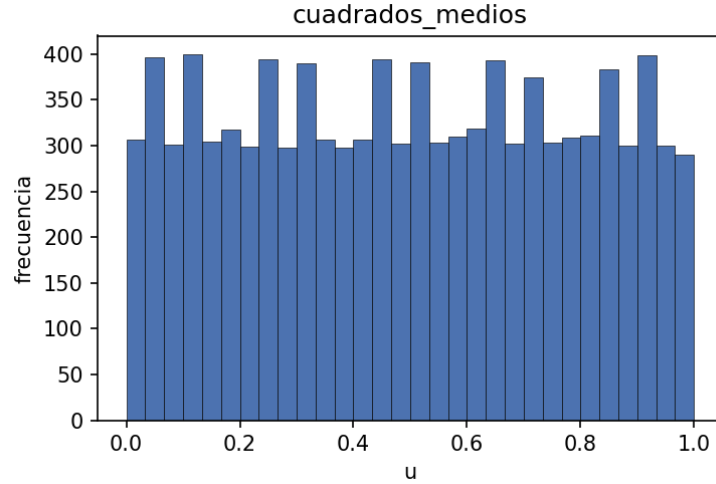


Figura 3: Histograma de 10,000 valores generados con cuadrados medios.

A simple vista el histograma parece razonablemente uniforme, pero muestra un patrón de peine (barras alternadamente más altas y más bajas). La prueba chi-cuadrado lo confirma de forma contundente: $\chi^2 = 152,94$, $p \approx 0,0000$, rechazando uniformidad con un margen enorme. La autocorrelación de retardo 1 también resultó significativa ($r = -0,039$, $p = 0,0001$). Esto es exactamente la falla clásica del método: la secuencia cae en ciclos cortos que repiten un subconjunto pequeño de valores muchas veces, lo que produce un histograma visualmente aceptable pero estadísticamente muy alejado de una uniforme real.

1.3. Mersenne Twister

El Mersenne Twister (variante MT19937) mantiene un estado interno de $n = 624$ palabras de 32 bits. En cada bloque de n salidas se aplica una recurrencia de "twist" sobre el estado, y cada palabra extraída pasa además por una transformación de temple (tempering) que mejora sus propiedades de equidistribución.

Algorithm 3 Mersenne Twister (MT19937), esquema general

Require: semilla x_0

```

1: Inicializar el arreglo de estado  $mt[0..623]$  a partir de  $x_0$  mediante un LCG auxiliar
2: for cada bloque de 624 salidas do
3:   for  $i = 0, \dots, 623$  do
4:      $y \leftarrow (mt[i] \& 0x80000000) + (mt[(i + 1) \bmod 624] \& 0x7fffffff)$ 
5:      $mt[i] \leftarrow mt[(i + 397) \bmod 624] \oplus (y \gg 1)$ 
6:     if  $y$  es impar then
7:        $mt[i] \leftarrow mt[i] \oplus 0x9908b0df$ 
8:     end if
9:   end for
10: end for
11:  $y \leftarrow mt[i]$ 
12:  $y \leftarrow y \oplus (y \gg 11)$ 
13:  $y \leftarrow y \oplus ((y \ll 7) \& 0x9d2c5680)$ 
14:  $y \leftarrow y \oplus ((y \ll 15) \& 0xefc60000)$ 
15:  $y \leftarrow y \oplus (y \gg 18)$ 
16: return  $y/2^{32}$ 

```

El período del Mersenne Twister es $2^{19937} - 1$, y su nombre viene precisamente de que ese número es un primo de Mersenne. La implementación se validó comparando la primera salida cruda con semilla 5489 contra el valor de referencia publicado, 3499211612, que coincidió exactamente.

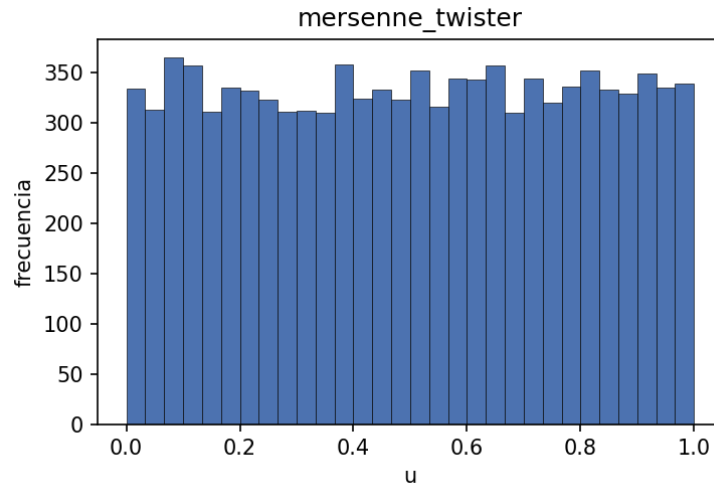


Figura 4: Histograma de 10,000 valores generados con Mersenne Twister.

Con semilla 5489 se obtuvo $\chi^2 = 12,14$, $p = 0,880$ para uniformidad, y $r = -0,0026$, $p = 0,797$

para independencia serial: ambas pruebas son plenamente consistentes con una secuencia uniforme e independiente.

1.4. Blum Blum Shub

Blum Blum Shub genera bits (o números) mediante $x_{n+1} = x_n^2 \bmod M$, donde $M = pq$ es producto de dos primos grandes, ambos congruentes con 3 mód 4 (un entero de Blum). Su seguridad depende de la dificultad de calcular residuos cuadráticos módulo M sin conocer su factorización, un problema tan difícil como factorizar M .

Algorithm 4 Blum Blum Shub

Require: semilla x_0 coprime con $M = pq$, $p \equiv q \equiv 3 \pmod{4}$

```

1: for  $n = 0, 1, 2, \dots$  do
2:    $x_{n+1} \leftarrow x_n^2 \bmod M$ 
3:    $u_{n+1} \leftarrow x_{n+1}/M$ 
4: end for
```

Para esta tarea se usaron los primos $p = 300007$ y $q = 300163$ (ambos $\equiv 3 \pmod{4}$), con $M = 90,051,001,141$ y semilla $x_0 = 987654323$. Estos primos son mucho más pequeños que los que se usarían en una aplicación criptográfica real (donde p y q tendrían cientos de dígitos); se eligieron de este tamaño únicamente para que generar miles de valores fuera rápido en Python puro.

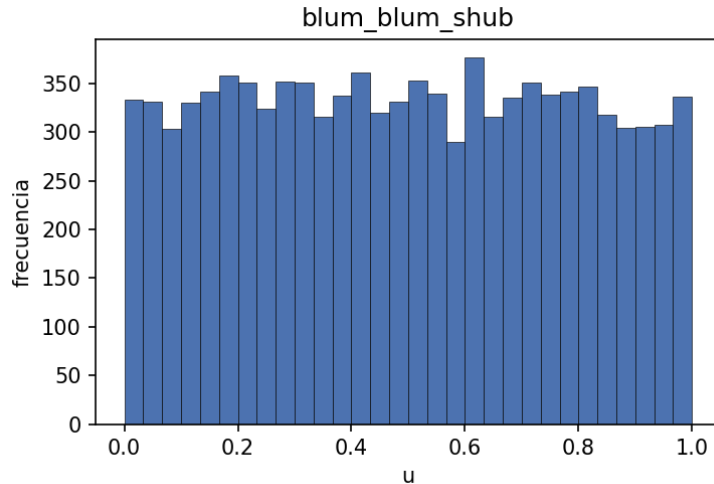


Figura 5: Histograma de 10,000 valores generados con Blum Blum Shub.

Se obtuvo $\chi^2 = 29,66$, $p = 0,056$ para uniformidad (justo por encima del umbral usual de 0.05) y $r = 0,0174$, $p = 0,083$ para independencia serial: no hay evidencia fuerte de desviación de la uniformidad ni de dependencia serial, aunque el p -valor de uniformidad queda cerca del límite.

1.5. RANDU

RANDU es un caso particular de LCG con $a = 65539 = 2^{16} + 3$, $c = 0$ y $m = 2^{31}$, usado en las computadoras IBM System/360 durante los años sesenta. Es históricamente célebre por lo mal elegido de su multiplicador.

Algorithm 5 RANDU

Require: semilla impar x_0

```
1: for  $n = 0, 1, 2, \dots$  do  
2:    $x_{n+1} \leftarrow (65539 \cdot x_n) \bmod 2^{31}$   
3:    $u_{n+1} \leftarrow x_{n+1}/2^{31}$   
4: end for
```

Debido a que $m = 2^{31}$ y $a - 1 = 65538$ no cumple la condición de Hull–Dobell relacionada con los factores de m , el período efectivo de RANDU con semilla impar es 2^{29} , es decir, un cuarto del máximo teórico m . La semilla usada fue $x_0 = 1$.

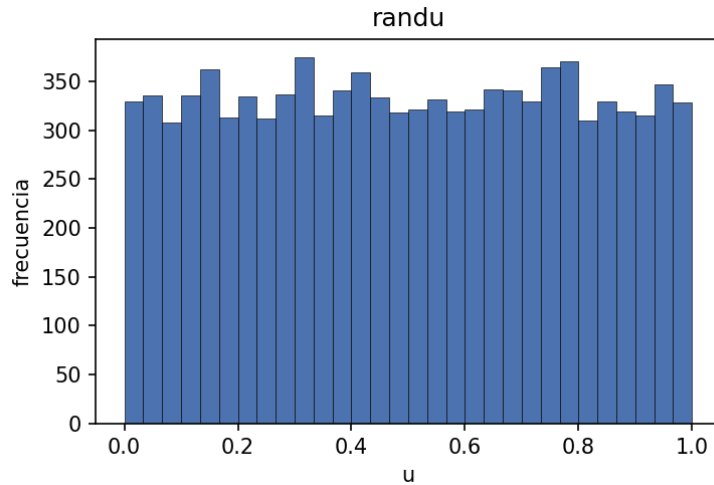


Figura 6: Histograma de 10,000 valores generados con RANDU.

En una dimensión, RANDU pasa las pruebas sin problema: $\chi^2 = 15,16$, $p = 0,712$ para uniformidad, y $r = 0,0133$, $p = 0,184$ para independencia serial. Esto ya es una lección importante: las pruebas de uniformidad e independencia serial de retardo 1 son *incapaces* de detectar el defecto real de RANDU, que solo aparece al observar ternas de valores consecutivos.

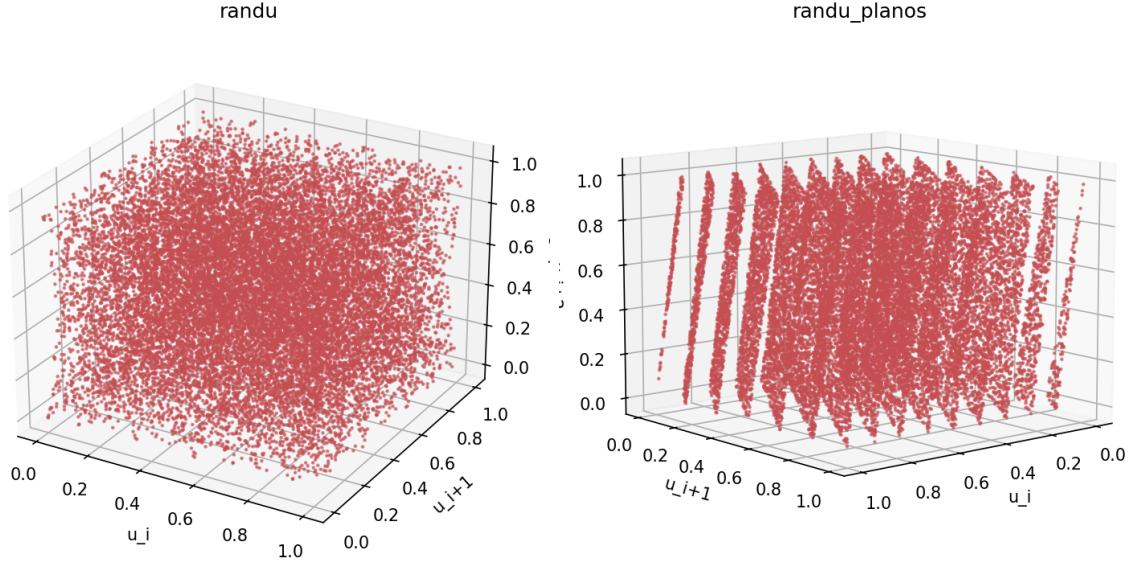


Figura 7: Ternas consecutivas de RANDU en $[0, 1]^3$. Izquierda: ángulo de vista genérico, donde el defecto no es evidente. Derecha: mismo conjunto de puntos visto desde el ángulo que deja ver de canto los hiperplanos sobre los que caen todas las ternas.

Puede demostrarse que toda terna consecutiva de RANDU satisface exactamente $x_{n+2} = 6x_{n+1} - 9x_n$ (mód 2^{31}), lo cual implica que los puntos (u_n, u_{n+1}, u_{n+2}) caen sobre la familia de planos paralelos $9u_n - 6u_{n+1} + u_{n+2} = k$ para k entero. Dentro del cubo $[0, 1]^3$ esa expresión solo puede tomar unos 15 valores enteros distintos, así que las ternas de RANDU quedan confinadas a apenas 15 hiperplanos, en vez de llenar el cubo. Desde un ángulo de vista genérico esto no salta a la vista, pero basta rotar la cámara para verlo con total claridad. Esta es la falla que Marsaglia documentó en *Random Numbers Fall Mainly in the Planes*.

1.6. Comparación de velocidad, período, seguridad y casos de uso

Algoritmo	Velocidad (núm/s)	Período	Seguro	Uso típico
LCG	$\sim 1,05 \times 10^7$	2^{32} (buenos parámetros)	No	Simulaciones, videojuegos
Cuadrados medios	$\sim 4,4 \times 10^6$	Corto e impredecible	No	Interés histórico/didáctico
Mersenne Twister	$\sim 1,9 \times 10^6$	$2^{19937} - 1$	No	Simulación científica, random
Blum Blum Shub	$\sim 1,0 \times 10^7$ *	Grande si p, q son primos seguros	Sí (criptográfico)	Claves, cifrado de flujo
RANDU	$\sim 1,2 \times 10^7$	2^{29} (defectuoso)	No	Ninguno (obsoleto)

Cuadro 1: Comparación de los cinco algoritmos. Las velocidades se midieron generando 100,000 números en la misma máquina. (*) La velocidad de Blum Blum Shub depende fuertemente del tamaño de los primos: aquí se usaron primos pequeños por rapidez, no primos criptográficamente seguros.

Mersenne Twister resulta más lento que LCG y RANDU porque cada salida involucra varias operaciones de temple además de la recurrencia de estado, mientras que LCG y RANDU son una sola multiplicación y suma módulo m . Blum Blum Shub, pese a ser el único diseñado con garantías criptográficas, aquí resulta rápido solo porque se usaron primos de unas pocas decenas de bits; con primos de tamaño criptográfico real (2048 bits o más) sería órdenes de magnitud más lento que los demás, ya que ese costo computacional es precisamente lo que sostiene su seguridad.

1.7. Comparación con las librerías estándar de Python

Se comparó la implementación propia de Mersenne Twister contra `random.Random` (que también usa MT19937 internamente, implementado en C) y contra `secrets.SystemRandom` (que obtiene aleatoriedad del sistema operativo, pensado para uso criptográfico). Se generaron 300,000 números con cada uno:

Generador	Tiempo (s)	χ^2	p -valor
Mersenne Twister propio	0.157	21.93	0.288
<code>random</code> estándar	0.010	26.14	0.126
<code>secrets</code> (CSPRNG)	0.193	33.59	0.021

Cuadro 2: Comparación de velocidad y bondad de ajuste, $N = 300,000$.

La versión de `random` de la librería estándar es unas 15 veces más rápida que la implementación propia porque su ciclo interno está escrito en C, mientras que la nuestra corre en Python puro instrucción por instrucción. `secrets` resulta el más lento de los tres, un costo esperado: para ser criptográficamente seguro no puede depender de un estado interno predecible como el de un Mersenne Twister, así que consulta al sistema operativo en cada llamada. El p -valor bajo de `secrets` en esta corrida particular no indica ningún defecto real, es simplemente la variabilidad normal de una prueba de hipótesis: repitiendo el experimento con otra semilla del sistema el resultado cambia.